

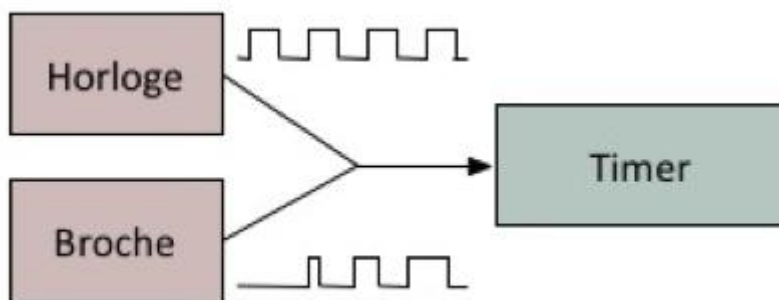
Gérer le temps avec les timers

Après avoir vu les principes de la configuration d'un périphérique, nous allons nous pencher sur un périphérique essentiel : le timer.

Principe des timers

Un timer est avant tout un compteur électronique . Il a une résolution qui dépend de son architecture et contraint l'amplitude des valeurs sur lesquelles il peut compter . Ainsi un timer avec une résolution de 8 bits pourra compter entre 0 à 255, un timer de 16 bits entre 0 à 65 535 et un timer de 32 bits de 0 à 4 294 967 295 ! A noter que dans de nombreuses architectures il est possible de « mettre en série » deux timers du processeur. Ainsi si on dispose de 2 timers 16 bits en les sérialisant, on peut fabriquer un timer 32 bits.

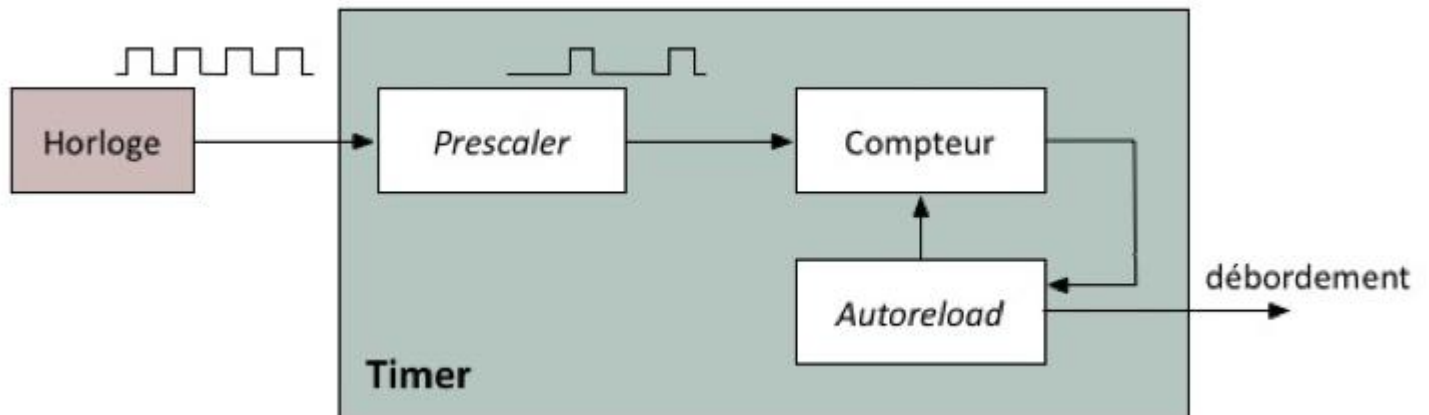
L'incrémentation d'un timer se fait sur des événements. Ces événements sont usuellement générés par une horloge avec une fréquence fixée. Dans ce cas, le timer “compte le temps” d'où son nom. Il est cependant possible de configurer le timer pour compter des événements qui ne sont pas périodiques, par exemple un signal venant d'une broche et présentant des fronts montants et descendants. Dans ce cas, on parle alors plutôt de compteur pour désigner le timer. Dans la suite, nous ne nous focaliserons que sur le comportement du timer dans le cas d'une horloge comme source d'événements.



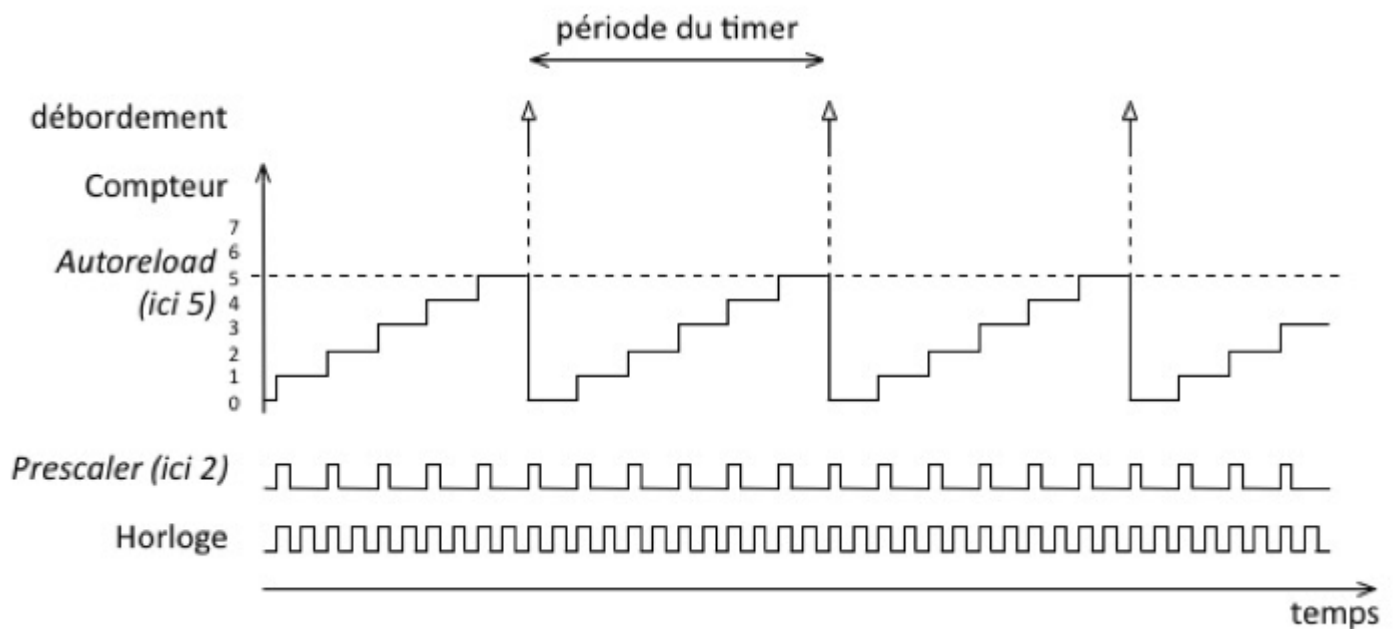
L'entrée d'horloge d'un timer est très souvent précédée d'un diviseur (*prescaler*). Son rôle est d'opérer une première division de la fréquence de l'horloge pour “ralentir” la fréquence de comptage du timer (nous verrons par la suite son intérêt pratique).

L'utilisation classique d'un timer va consister à réagir au débordement (*overflow*) de son compteur. En effet, la résolution du timer étant bornée, son incrémentation va inévitablement

conduire à un débordement, c'est-à-dire que l'incrément suivant dépasse la capacité du compteur. Lorsque ce débordement survient, le compteur est remis à zéro et un événement interne est généré pour signaler cette rupture dans la séquence de comptage.



Afin de mieux contrôler le débordement, le compteur est associé à un registre dit *autoreload*. Celui-ci contient la valeur à laquelle le compteur va déborder, même s'il n'a pas atteint sa capacité maximale. Ainsi, le compteur va avoir le comportement décrit par la figure ci-dessous :



On voit que le timer est incrémenté à chaque impulsion de l'horloge et que lorsque son compteur atteint la valeur de l'*autoreload*, il déborde.

Calculer les paramètres d'un timer

En connaissant la fréquence de l'horloge entrante du timer et en réglant le *prescaler* et l'*autoreload* il est possible de régler finement la fréquence à laquelle le timer déborde.

La formule pour obtenir la période à laquelle le timer déborde est donnée par :

$$T_{timer} = T_{horloge} \times prescaler \times (autoreload + 1)$$

Supposez que vous avez une horloge à 72 Mhz, comment régler le *prescaler* et l'*autoreload* pour obtenir un débordement du timer toutes les secondes ?

Considérons le cas où le *prescaler* vaut 1, nous avons alors :

$$autoreload = \frac{T_{timer}}{T_{horloge}} - 1 = \frac{1}{\frac{1}{72.10^6}} - 1 = 72.10^6 - 1$$

ce qui serait possible pour un timer comptant sur 32 bits, mais qui devient impossible pour un timer en 16 bits puisqu'il ne peut compter que jusqu'à 65 535.

Pour configurer un timer, il faut donc connaître sa résolution et faire attention à l'amplitude des valeurs disponibles.

Pour trouver une configuration faisable, Il faut aussi utiliser le *prescaler*.

Dans notre exemple, en fixant le *prescaler* à 7 200, nous avons

$$autoreload = \frac{T_{timer}}{(prescaler \times T_{horloge})} - 1 = \frac{1}{\frac{7200}{72.10^6}} - 1 = 1.10^4 - 1$$

qui est une valeur acceptable pour un compteur 16 bits.

Au final, il est donc possible de configurer le *prescaler* à 7 200 et l'*autoreload* à 9 999 pour avoir un timer qui expire toutes les secondes (dans le cas d'un compteur en 16 bits). D'autres valeurs sont bien évidemment possibles.

Remarquez cependant qu'il peut ne pas y avoir de solution entière pour les valeurs du *prescaler* et de l'*autoreload*. Dans ce cas, il vaut mieux chercher à régler ces valeurs de manière à minimiser l'erreur induite par les arrondis. En prenant un prescaler le plus petit possible on a tendance à minimiser les erreurs puisque l'unité de comptage (la période de comptage du timer après la division par le *prescaler*) sera plus petite.

Configurer un timer sur le STM32F103

Passons à la pratique pour le micro-contrôleur STM32F103. D'après la documentation, nous apprenons qu'il y a plusieurs timers disponibles et qu'ils sont de deux espèces différentes : la première est constituée des timers 1 et 8 qui sont des *Advanced-control timers* alors que les timers 2 à 5 sont désignés comme étant des *General-purpose timers*. Nous ne verrons pas ici la différence entre ces deux espèces et nous focaliserons sur les timers 2 à 5. Il existe également un timer spécifique au cœur du cortex appelé SysTick. Il est globalement plus simple que les autres et a pour vocation d'être utilisé par un système d'exploitation. Nous ne le détaillerons pas non plus dans ce chapitre.

La carte nucléo est équipée d'un STM32F103RB pour lequel tous les timers ne sont pas présents. Il contient uniquement l'advanced timer 1 et les general purpose timers 2, 3 et 4. Pour la suite, on suppose que les timers sont branchés sur une horloge à 72 MHz, ce qui est par défaut la configuration de la carte Nucléo.

Enfin, parmi les registres de configuration d'un timer nous trouvons *ARR* pour l'*autoreload* et *PSC* pour le *prescaler*. Remarquons au passage que le prescaler divise par 1 pour la valeur 0, par 2 pour la valeur 1, etc. Il faut donc dans la formule précédente considérer que

$$T_{timer} = T_{horloge} \times (PSC + 1) \times (ARR + 1)$$

Nous allons configurer le timer 2, désigné par TIM2 dans le fichier *stm32f10x.h* et commençons avec un projet contenant uniquement un main vide.

```
#include <stm32f10x.h>

int main (void)

{

    RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;

    while(1)

    {

    }

    return 0;

}
```

Ne vous focalisez pas sur la ligne

```
RCC->APB1ENR |= RCC_APB1ENR_TIM2EN;
```

elle permet de configurer les horloges (nous verrons plus loin plus en détail les horloges sur le STM32).

Pour configurer le timer 2, il suffit donc d'ajouter avant la bouclewhile les lignes :

```
TIM2->ARR = 9999;

TIM2->PSC = 7199;
```

Il ne vous reste plus qu'à démarrer le timer. Pour cela, la documentation (p. 404) indique que le bit 0, nommé CEN, du registre CR1 permet de lancer et d'arrêter le compteur. Il faut donc ajouter

```
TIM2->CR1 = TIM2->CR1 | ( 1 << 0);
```

pour lancer le compteur. Remarquez que le masque du bit CEN est défini dans le fichier *stm32f10x.h* et qu'il est possible d'écrire directement

```
TIM2->CR1 = TIM2->CR1 | TIM1_CR1_CEN;
```

Compiler et passer en debug pour vérifier votre configuration. Mettez un point d'arrêt sur la ligne `TIM2->CR1 |= TIM1_CR1_CEN` et ouvrez la fenêtre liée au timer 2 dans *Peripheral*. Vous devriez observer que les registres ARR et PSC prennent bien les valeurs 0x270F (pour 9999) et 0x1C1F (pour 7199).

Avancez le simulateur pour lancer le comptage et observez que le compteur, représenté par le registre CNT, s'incrémente bien.

Scruter le débordement d'un timer

Une manière simple de réagir au débordement d'un timer est de guetter, on parle plutôt de scruter, l'événement provoqué par ce débordement. Pour cela, il existe un registre contenant un bit indiquant si un débordement a eu lieu ou non.

Pour le STM32F103, le débordement est indiqué par la valeur du bit 0, nommé UIF, du registre SR (voir documentation p. 409). Suite à un débordement ce bit passe à 1.

Une manière de « guetter » l'instant où le timer déborde consiste donc à scruter dans une boucle la valeur du bit UIF.

Pour mettre cela en œuvre, nous allons faire clignoter la led USER toutes les secondes. Pour rappel cette led est branchée sur la broche 5 du port A. Afin de la contrôler, il faut la configurer en sortie soit ajouter avant la boucle le code

```
RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

GPIOA->CRL &= ~(0xF << 20); // PA.5 output push-pull

GPIOA->CRL |= (0x1 << 20); // PA.5 output push-pull
```

Nous allons maintenant ajouter dans la boucle while le code pour scruter l'état du timer

```
if (TIM2->SR & TIM_SR_UIF) { // Scrutation de UIF

    TIM2->SR = TIM2->SR & ~TIM_SR_UIF; // Remise à zéro de UIF

}
```

Remarquez d'une part que le masque TIM_SR_UIF existe déjà et vaut 0x1 et d'autre part que nous avons dû remettre à zéro le bit UIF. Cela est nécessaire sinon il reste dans l'état indiquant un débordement et à la prochaine évaluation dans la boucle le timer sera toujours considéré comme ayant débordé.

Pour faire clignoter la led, il suffit d'ajouter juste après

```
TIM2->SR = TIM2->SR & ~TIM_SR_UIF
```

une opération XOR bit à bit entre 1 et le bit 5 du registre ODR du port GPIOA, soit :

```

if (TIM2->SR & TIM_SR_UIF) {

    TIM2->SR = TIM2->SR & ~TIM_SR_UIF;

    GPIOA->ODR = GPIOA->ODR ^ (1 << 5);

}

```

Compilez, lancez la simulation et observez l'état de la broche 5 du port A. Vous devriez la voir clignoter toutes les secondes (en temps de simulation cela devrait prendre un peu plus).

Maintenant passez en réel et testez votre code sur la carte Nucleo : vous devriez observer la led verte qui clignote.

Nous verrons dans le chapitre suivant un moyen bien plus efficace de guetter le débordement d'un timer. Mais avant d'aborder ce point, intéressons-nous aux horloges.

Principe des arbres d'horloge

Jusqu'ici nous avons passé sous silence la raison pour laquelle il faut ajouter dans le code une ligne du type `RCC->APB2ENR |= RCC_APB2ENR_IOPAEN`. Cette ligne a pour but d'activer les horloges reliées aux différents périphériques. Si cela n'est pas fait, les périphériques n'ont pas de signal pour faire évoluer leurs circuits logiques et ne sont donc pas actifs.

Mais pourquoi les horloges ne sont pas actives par défaut ?

La raison de ce système est d'économiser l'énergie consommée par le circuit. En désactivant les horloges des circuits, ceux-ci ne sont plus actifs et donc ne consomment rien. Si les horloges étaient actives, les circuits auraient été sollicités et donc consommeraient un peu, même s'ils ne font rien...

Il faut donc absolument allumer les horloges pour les périphériques que l'on utilise avant de commencer à les configurer.

Par exemple, le code

```

GPIOA->CRL &= ~(0xF << 4*5); // PA.5 output push-pull

GPIOA->CRL |= (0x1 << 4*5); // PA.5 output push-pull

RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

```

ne permet pas de configurer le port car l'horloge a été activée après et donc les registres du périphériques ne sont pas mis à jour.

Un autre élément qu'il faut appréhender pour bien maîtriser l'écoulement du temps sur un microcontrôleur est la source d'horloge du système. Dans le cas du STM32, mais qui est aussi commun à d'autres familles de processeurs, il est possible d'avoir diverses sources pour l'horloge globale du système.

Par défaut, le microcontrôleur dispose d'un circuit RC qui oscille à 8MHz, lui permettant ainsi d'avoir sa propre horloge et donc de fonctionner par défaut. L'inconvénient d'un tel circuit est qu'il est très sensible aux conditions externes (vous pouvez par exemple utiliser une bombe à froid pour perturber l'oscillation du circuit).

Il est possible de rendre plus robuste cette horloge en ajoutant un quartz externe au microcontrôleur. Dans le cas de la Nucleo, il existe un quartz externe cadencé à 8MHz qui permet, une fois l'arbre d'horloge correctement configuré de faire tourner le cœur à une fréquence de 72MHz (l'horloge du système est nommée SYSCLK).

A cela s'ajoute un grand nombre de circuits permettant de régler finement les horloges dans le microcontrôleur. La documentation page 92 donne un aperçu de ce qui s'appelle l'arbre des horloges. On remarque sur cette figure que l'entrée principale SYSCLK est configurée par défaut à 72Mhz.

Ce signal peut ensuite être divisé à l'aide du *prescaler* AHB qui servira ensuite d'entrée pour les deux sous-arbres APB1 et APB2 qui alimentent les périphériques tels que les timers.

Il est donc possible de configurer les registres des *prescaler* de AHB, APB1 et APB2 pour faire battre les périphériques à des rythmes plus faibles et ainsi économiser l'énergie globalement consommée par le microcontrôleur. Pour la suite, nous ne toucherons à rien de tout cela et garderons les configurations initiales. Par contre, n'oubliez pas d'activer les horloges des périphériques avant de les utiliser.